Study and Implementation of Git

Ansh Jindal
Department of Computer Science and Engineering
Manipal Institute of Technology, MAHE
Manipal,India
ansh10.mitmpl2026@learner.manipal.edu

Abstract – This report details the study of the Git version control system. Core concepts such as commits, branches, merging, rebasing, and remote repositories were explored using tutorials from the Learn Git Branching website. These principles were subsequently applied to a local demonstration repository containing problem solution files.

I. INTRODUCTION

Software projects require rigorous tracking of changes throughout their development lifecycle. Git, a globally utilized distributed version control system, provides a structured methodology to maintain project history. By keeping local repositories, Git allows developers to execute operations without a continuous connection to a remote server. Essential features include cloning, committing, pushing, pulling, branching, merging, and rebasing.

II. GIT COMMITS AND HISTORY

a. *GIT Commits*

A commit represents a saved version of a project at a particular point in time. Each commit has a reference to a previous commit which enables git to record the history of changes made to a project. When a new commit is created the current branch moves to the new commit and every new commit contains a hardcoded pointer to its parent commit. This is what allows Git to maintain history of the project without losing previous versions.

b. *HEAD and Relative References*

Git uses "HEAD" to refer to the currently checked-out branch, usually pointing to the latest commit. Git also supports relative references like HEAD^ and HEAD~1 to navigate backward through commits without relying on full alphanumeric commit hashes.

III. BRANCHING AND BRANCH MANAGEMENT

a. ***The Nature of a Branch:*** *A branch (like main) is not a duplicate of the repository; it is simply a small file containing the hash code of a specific commit. Creating a branch via git branch or git checkout -b is extremely fast because it only involves generating this pointer file.*

b. *Merging*

Merging integrates changes from one branch into another. If the receiving branch hasn't diverged, Git performs a fast-forward merge by simply moving the pointer forward. If both branches possess unique commits, Git executes a three-way merge by comparing the last common ancestor and creating a new commit with two parents.

c. *Rebasing*

Rebasing takes unique commits from a current branch and reapplies them sequentially on top of another branch. While this creates a linear history without merge commits, it changes the hash values of the commits. Consequently, rebasing must not be applied to branches that have already been pushed and shared with collaborators.

IV. REMOTE REPOSITORIES : PUSH & PULL

a. *Tracking branching and origin*

A remote repository shortcut is typically named origin. It includes remote tracking references that log the state of branches during the last communication with the remote server.

b. *Fetch vs Pull*

The git fetch command safely downloads new commits and updates remote references without altering the local working directory. In contrast, git pull fetches data and immediately merges the updated remote branch into the current local branch, which can sometimes result in merge conflicts.

c. *Force Pushing*

Standard pushes only succeed if they append commits on top of what the remote already has. If local history was altered via rebasing, a standard push is rejected. Using git push --force overrides the remote branch, but must be used with caution as it can delete commits pulled by others..

V.HISTORY, LOGS AND THE REFERENCE LOG

a. *git log and Commit Ancestry*

The **git log** command is used to view the commit history of a Git repository. By default, it lists the commits made in your current branch in **reverse chronological order** (newest commits first), displaying the unique commit SHA hash, the author, the date, and the commit message

b. *git reflog*

git reflog is your local Git safety net. It acts like an automated diary or a "black box flight recorder", keeping track of every single time your **HEAD pointer moves** in your repositorybehind instead of building off of them. reflog entries only exist locally and are never pushed to a remote.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

Whether you switched branches, committed code, performed a rebase, or did a hard reset, git reflog records it. This makes it the ultimate "undo button" when you think you have permanently lost work

VI.RESOLVING MERGE CONFLICTS

A merge conflict happens when Git cannot automatically combine two branches during a merge because both of them have changes made to the same line of the same file. When this happens, Git stops and marks the conflicting part of the file with <<<<<<< HEAD, then ======= , and then >>>>>>> followed by the branch name.

To fix a conflict, the file has to be opened and edited manually. After that, `git add` is run on the file to mark it as resolved, and git commit is run to finish the merge.

STASHING

The git stash command temporarily saves uncommitted modifications, allowing developers to switch branches (e.g., to fix an urgent bug) in a clean environment. Changes can be restored later using git stash pop or git stash apply

VII. DEMONSTRATION : LEARN GIT BRANCHING AND A LOCAL REPOSITORY

a. A little about the Main and Remote tabs on Learn Git Branching

The Main tab helped me understand the basic Git commands and how commits and branches are managed on local repositories whereas the Remote tab gave me a better understanding of pushing, pulling, and working with remote branches. I found the difference between merge and rebase especially useful, as they result in different commit histories. I also found the diverged remote exercises useful, especially seeing how Git can reject a push when the local and remote histories are no longer in sync.

b. Local Repository setup and Task 1 integration

I created a repository `git init` inside a folder containing the eight Task 1 Python solution files. All the files were staged with `git add .` and recorded in a single initial commit using `git commit`.

I then created an empty repository GitHub and linked to the local repository as origin using `git remote add origin`, after which the initial commit was pushed using `git push -u origin main`.

c. Demonstrating Commit, Branch, and Merge

I created a new branch named feature-update using `git checkout -b`. I then added a feature to the Task 1 Question 1 python file (output dictionary arranges the output alphabets in alphabetical order) and committed on this branch. Switching back to main and running `git merge feature-update` completed the merge. Since main had not changed since the branch was created, this resulted in a fast-forward merge, meaning Git just moved the main branch pointer forward with no separate merge commit created, as explained in Section III.B.

d. Demonstrating a Merge Conflict and Resolution

To simulate a merge conflict, a conflict-branch was created. The first line of a file was altered on this branch, while the exact same line was modified differently on main. Attempting to merge triggered a conflict. The file was manually sanitized of its conflict markers, staged, and committed to complete the merge.Running git merge conflict-branch on main produced a conflict, and Git marked the file with <<<<<<<, =======, and >>>>>>> markers as shown below.

The file was manually edited to remove the markers and keep the intended line, and the conflict was marked resolved using git add followed by git commit, completing the merge.

e. Demonstrating Push and Pull

After the merge, I pushed the main updated branch to the remote using `git push origin main`. To demonstrate a pull, a small change was made directly on GitHub's website (random test comment) and committed there, so that the remote repository had a commit the local repository did not yet have. When I ran `git pull origin main`, locally it fetched this new commit and merged it into the local main branch. Since the local branch had not diverged from the remote at that point, this was a fast-forward pull, and the change appeared immediately in the local files.

f. Log and Reflog Evidence

listing every checkout, commit, and merge performed during this demonstration in the order it happened, each entry referenced as HEAD@{n}. Comparing this with the output of `git log --oneline --graph --all`, shown earlier, illustrates the distinction discussed in Section V: `git log` shows only the permanent ancestry of commits reachable from the current branch, whereas, `git reflog` shows the complete local movement history of HEAD, including any actions that did not add new ancestry to the branch.

IX. CONCLUSION

This comprehensive exercise illustrated that Git operates not just as a collection of terminal commands, but as an intricate system of commits, branches, and dynamic pointers. Utilizing Learn Git Branching provided a visual foundation, while practical application on a local GitHub-linked repository solidified the realities of collaborative software development and manual merge conflict resolution.

ACKNOWLEDGMENT

Sincere thanks to the Driverless Subsystem seniors for their guidance and resources, which proved invaluable in mastering these version control workflows.

REFERENCES

[1] Learn Git Branching. [Online]. Available: https://learngitbranching.js.org/

[2] Git, “Git Reference” [Online]. Available: https://git-scm.com/docs